

# CONTROL SYSTEM LAB

## EEE-404

OPEN-ENDED LAB

PREPARED BY SOWRAV CHOWDHURY  
183016008

University of Liberal Arts Bangladesh

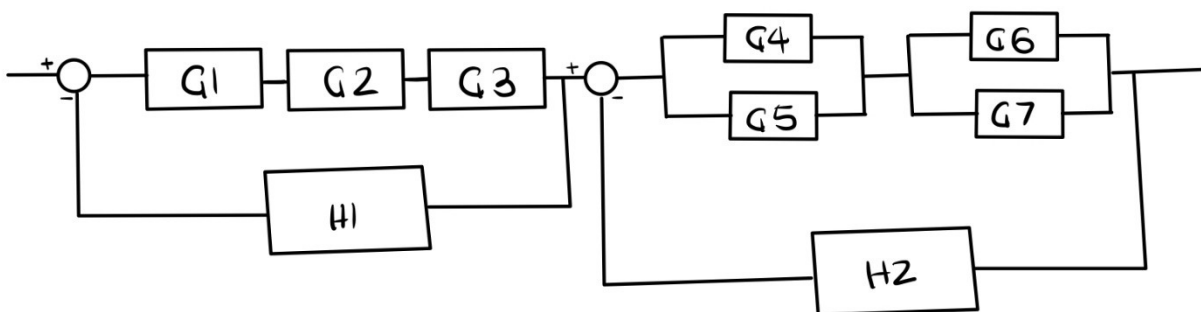
**Design a control system block diagram with the following conditions:**

System must have

- i) Three cascaded subsystems.
- ii) Two parallel subsystems
- iii) Two feedback systems (feedback form)
- iv) Two summing junctions

Also,

- i. Reduce the block diagram to a single transfer function.
- ii. Simulate and observe the output of the transfer function (with PID controller) using Simulink MATLAB.

**The control system block diagram developed is shown below:****The values of the blocks are:**

|                                                |                                                     |                                                 |
|------------------------------------------------|-----------------------------------------------------|-------------------------------------------------|
| <b>sysG1 =</b><br>$\frac{1}{s+2}$              | <b>sysG4 =</b><br>$\frac{2s^2 + s + 2}{s+2}$        | <b>sysG7 =</b><br>$\frac{2s+4}{s+2}$            |
| <b>sysG2 =</b><br>$\frac{s^2+1}{s^2+2s+1}$     | <b>sysG5 =</b><br>$\frac{s^3+s^2+s+1}{s^3+s^2+s+1}$ | <b>sysH1 =</b><br>$\frac{s^2+2s+3}{s^2+2s+1}$   |
| <b>sysG3 =</b><br>$\frac{s^2+2s+2}{2s^2+2s+1}$ | <b>sysG6 =</b><br>$\frac{2s+11}{s+1}$               | <b>sysH2 =</b><br>$\frac{2s^2+2s+21}{s^2+2s+3}$ |

**The transfer functions of the systems were developed in MATLAB using:**

```
numG1 = [1];
denG1 = [1 2];
sysG1 = tf(numG1, denG1)
```

```
numG2 = [1 0 1];
denG2 = [1 2 1];
sysG2 = tf(numG2, denG2)
```

```
numG3 = [1 2 2];
denG3 = [2 2 1];
sysG3 = tf(numG3, denG3)
```

```
numG4 = [2 1 2];
denG4 = [1 2];
sysG4 = tf(numG4, denG4)
```

```
numG5 = [1 1 1 1];
denG5 = [1 1 1 1];
sysG5 = tf(numG5, denG5)
```

```
numG6 = [2 0 1 1];
denG6 = [1 1];
sysG6 = tf(numG6, denG6)
```

```
numG7 = [2 4];
denG7 = [1 2];
sysG7 = tf(numG7, denG7)
```

```
numH1 = [1 2 3];
denH1 = [1 2 1];
sysH1 = tf(numH1, denH1)
```

```
numH2 = [2 2 2 1];
denH2 = [1 2 3];
sysH2 = tf(numH2, denH2)
```

***Series Connections were made using:***

```
SYS_SERIES_1 = series(sysG1,sysG2)
```

```
SYS_SERIES_2 = series(SYS_SERIES_1,sysG3)
```

```
SYS_SERIES_3 = series(SYS_SERIES_1,SYS_PARALLEL_2)
```

```
SYS_SERIES_4 = series(SYS_FDBCK_1,SYS_FDBCK_2)
```

***Parallel Connections were made using:***

```
SYS_PARALLEL_1 = parallel(sysG4,sysG5)
```

```
SYS_PARALLEL_2 = parallel(sysG6,sysG7)
```

**Feedback Connections were made using:**

```
SYS_FDBCK_1 = feedback(SYS_SERIES_2,sysH1)
```

```
SYS_FDBCK_2 = feedback(SYS_SERIES_3,sysH2)
```

**Complete code for reduction:**

```
close all;
clear all;
clc;

numG1 = [1];
denG1 = [1 2];
sysG1 = tf(numG1, denG1)

numG2 = [1 0 1];
denG2 = [1 2 1];
sysG2 = tf(numG2, denG2)

numG3 = [1 2 2];
denG3 = [2 2 1];
sysG3 = tf(numG3, denG3)

numG4 = [2 1 2];
denG4 = [1 2];
sysG4 = tf(numG4, denG4)

numG5 = [1 1 1 1];
denG5 = [1 1 1 1];
sysG5 = tf(numG5, denG5)

numG6 = [2 0 1 1];
denG6 = [1 1];
sysG6 = tf(numG6, denG6)

numG7 = [2 4];
denG7 = [1 2];
sysG7 = tf(numG7, denG7)

numH1 = [1 2 3];
denH1 = [1 2 1];
sysH1 = tf(numH1, denH1)

numH2 = [2 2 2 1];
denH2 = [1 2 3];
sysH2 = tf(numH2, denH2)

SYS_SERIES_1 = series(sysG1,sysG2)

SYS_SERIES_2 = series(SYS_SERIES_1,sysG3)

SYS_FDBCK_1 = feedback(SYS_SERIES_2,sysH1)

SYS_PARALLEL_1 = parallel(sysG4,sysG5)
```



```

SYS_PARALLEL_2 = parallel(sysG6,sysG7)

SYS_SERIES_3 = series(SYS_SERIES_1,SYS_PARALLEL_2)

SYS_FDBCK_2 = feedback(SYS_SERIES_3,sysH2)

SYS_SERIES_4 = series(SYS_FDBCK_1,SYS_FDBCK_2)

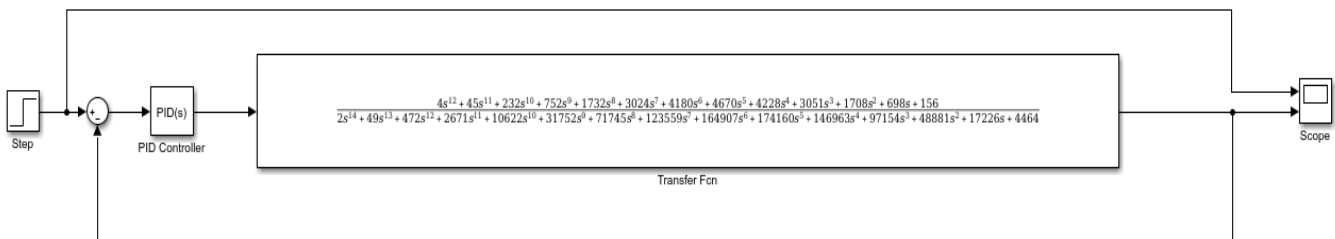
```

**The final Transfer Function we achieved was:**

`SYS_SERIES_4 =`

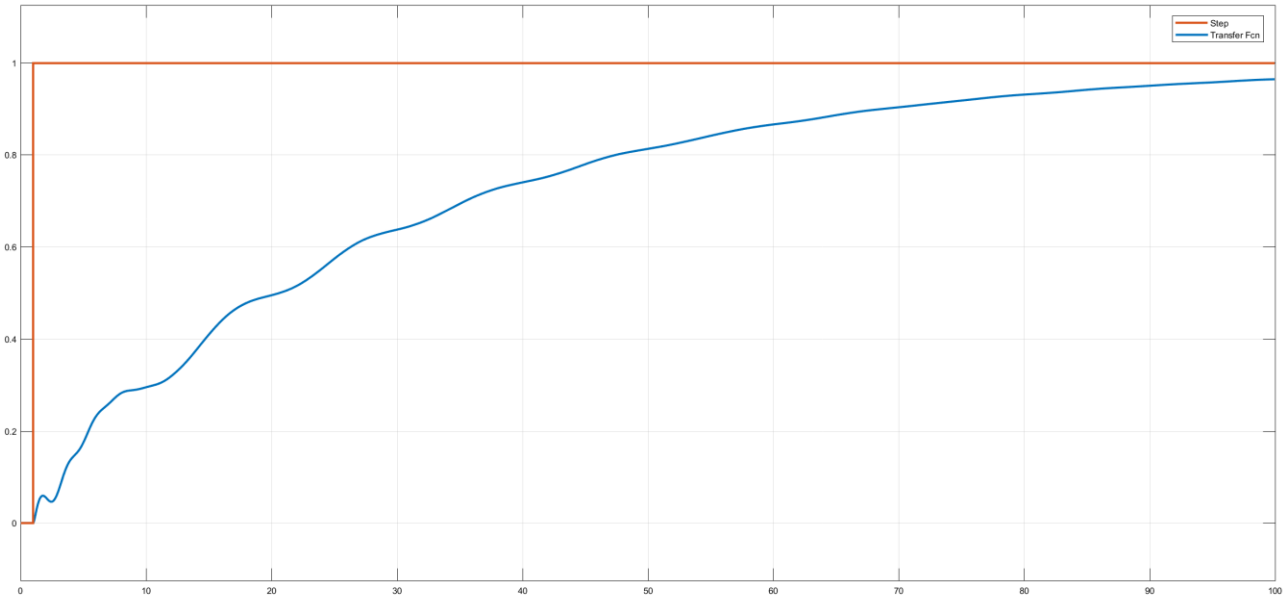
$$\frac{4s^{12} + 45s^{11} + 232s^{10} + 752s^9 + 1732s^8 + 3024s^7 + 4180s^6 + 4670s^5 + 4228s^4 + 3051s^3 + 1708s^2 + 698s + 156}{2s^{14} + 49s^{13} + 472s^{12} + 2671s^{11} + 10622s^{10} + 31725s^9 + 71745s^8 + 123559s^7 + 164907s^6 + 174160s^5 + 146963s^4 + 97154s^3 + 48881s^2 + 17226s + 4464}$$

Simulink Block Diagram for the above system:

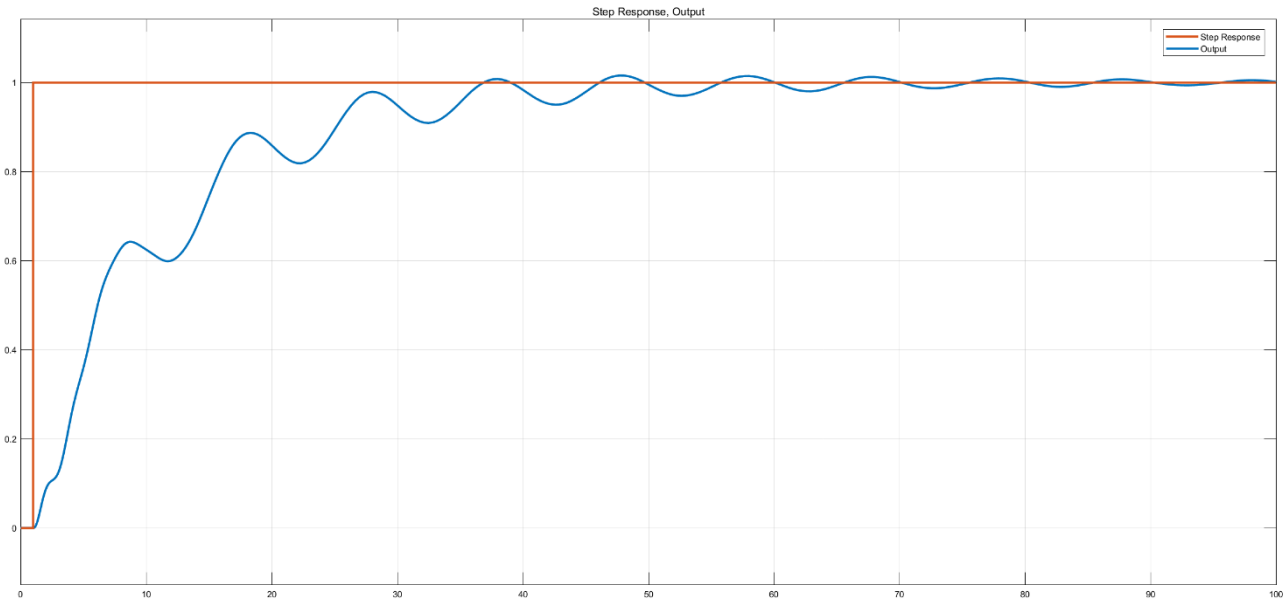


Step input was used to see the response, and then a summing junction is used to find the error signal in the closed-loop form. A PID block is used to tune and the transfer function as calculated from block reduction is used to see the response with scope.

Untuned closed-loop step response of the system simulated for 100s:



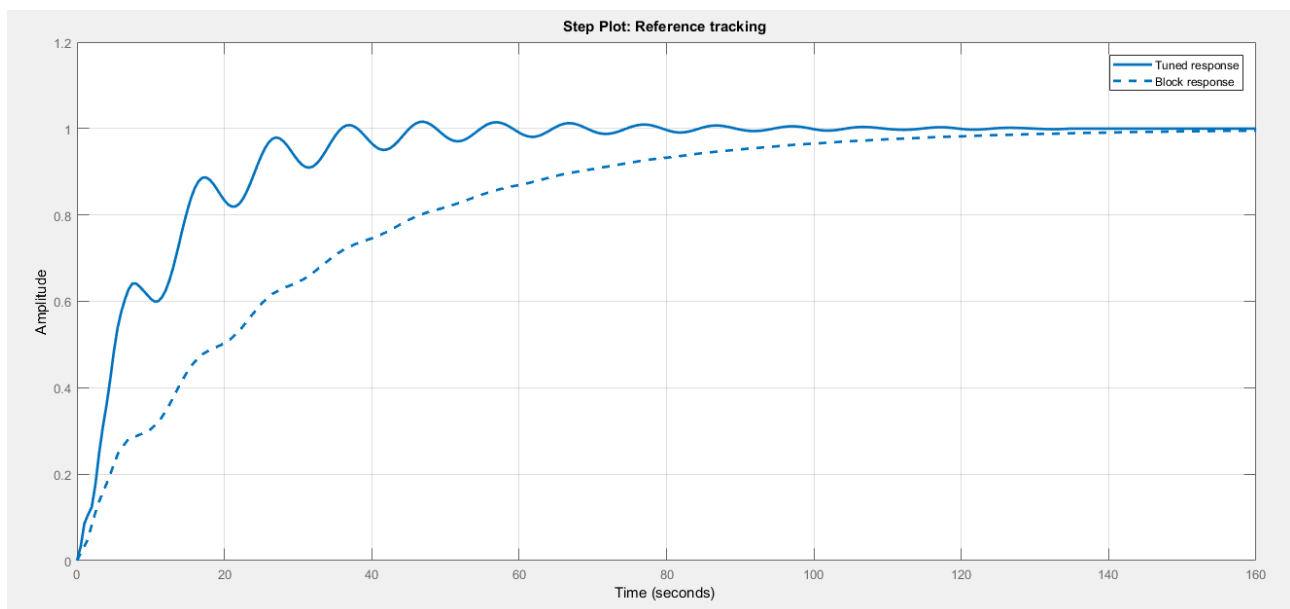
MATLAB Autotuned closed-loop step response of the system simulated for 100s:



**Parameters from MATLAB Autotune from PID Toolbox:**

| Controller Parameters |        |       |
|-----------------------|--------|-------|
|                       | Tuned  | Block |
| P                     | 0      | 1     |
| I                     | 2.9089 | 1     |
| D                     | 0      | 0     |
| N                     | 100    | 100   |
|                       |        |       |
|                       |        |       |

| Performance and Robustness |                        |                         |
|----------------------------|------------------------|-------------------------|
|                            | Tuned                  | Block                   |
| Rise time                  | 22.8 seconds           | 65.5 seconds            |
| Settling time              | 53.2 seconds           | 116 seconds             |
| Overshoot                  | 1.57 %                 | 0 %                     |
| Peak                       | 1.02                   | 0.999                   |
| Gain margin                | 6.67 dB @ 0.652 rad/s  | 28.7 dB @ 0.821 rad/s   |
| Phase margin               | 93.6 deg @ 0.105 rad/s | 93.2 deg @ 0.0351 rad/s |
| Closed-loop stability      | Stable                 | Stable                  |

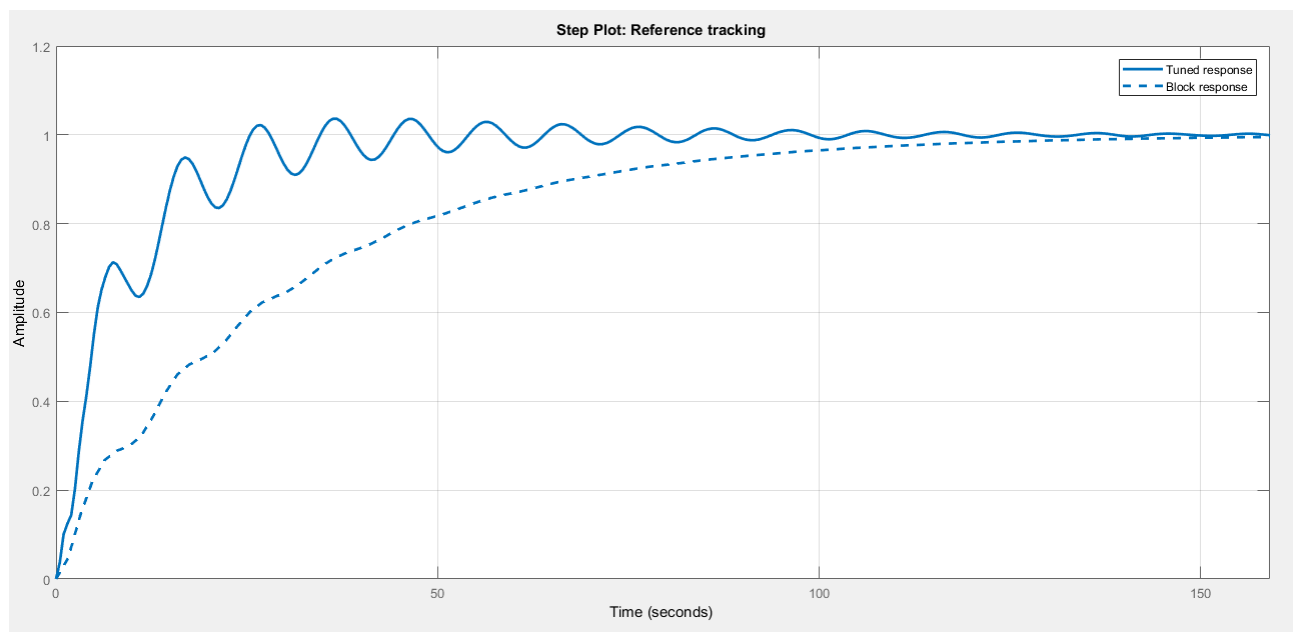
**Original (Block) vs MATLAB Autotuned (Tuned) System**

**Parameters from MATLAB self tuned from PID Toolbox:**

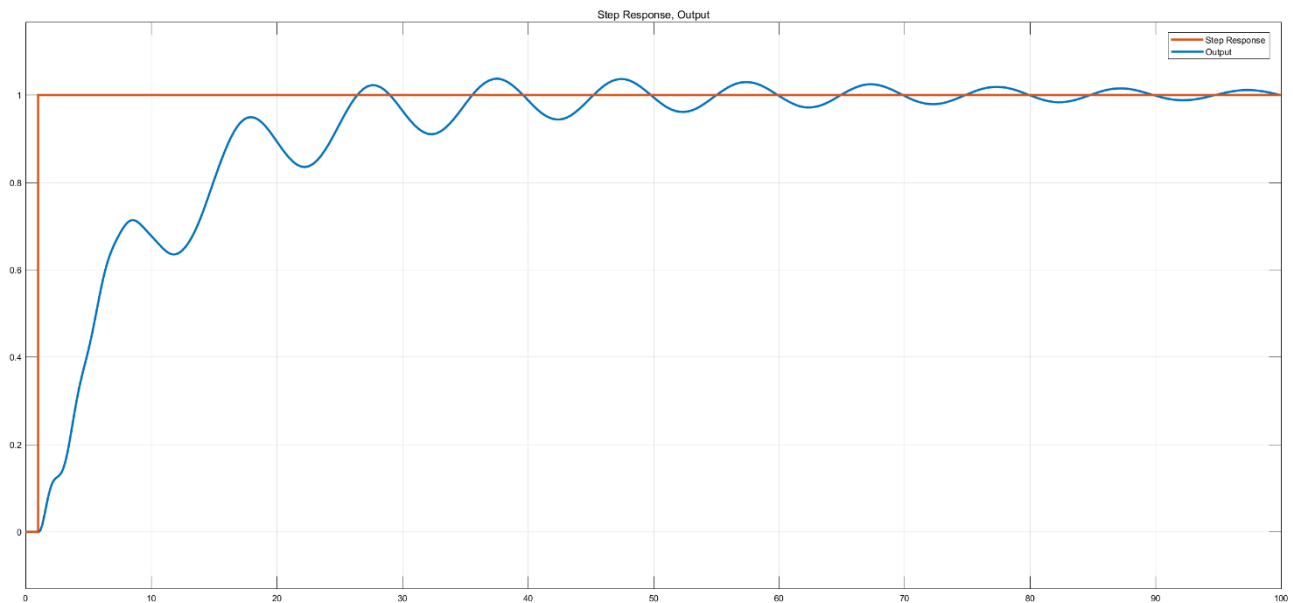
| Controller Parameters |        |       |
|-----------------------|--------|-------|
|                       | Tuned  | Block |
| P                     | 0      | 1     |
| I                     | 3.4528 | 1     |
| D                     | 0      | 0     |
| N                     | 100    | 100   |
|                       |        |       |
|                       |        |       |

| Performance and Robustness |                        |                         |
|----------------------------|------------------------|-------------------------|
|                            | Tuned                  | Block                   |
| Rise time                  | 14.3 seconds           | 65.5 seconds            |
| Settling time              | 71.8 seconds           | 116 seconds             |
| Overshoot                  | 3.65 %                 | 0 %                     |
| Peak                       | 1.04                   | 0.999                   |
| Gain margin                | 5.18 dB @ 0.652 rad/s  | 28.7 dB @ 0.821 rad/s   |
| Phase margin               | 94.2 deg @ 0.126 rad/s | 93.2 deg @ 0.0351 rad/s |
| Closed-loop stability      | Stable                 | Stable                  |

**Original (Block) vs MATLAB self tuned (Tuned) System**



**MATLAB Autotuned closed-loop step response of the system simulated for 100s:****Discussion:**

A block diagram provides a means to easily identify the functional relationships among the various components of a control system. The use of block diagrams to illustrate a cause-and-effect relationship is prevalent in control. We use operational blocks to represent transfer functions and lines for unidirectional information transmission. It is a nice way to visualize the interrelationships of various components. They will be crucial in helping us identify manipulated and controlled variables and input(s) and output(s) of a system.

**Block Diagram Reduction Rules**

these rules for simplifying (reducing) the block diagram, which is having many blocks, summing points and take-off points.

- Rule 1 – Check for the blocks connected in series and simplify.
- Rule 2 – Check for the blocks connected in parallel and simplify.
- Rule 3 – Check for the blocks connected in feedback loop and simplify.
- Rule 4 – If there is difficulty with take-off point while simplifying, shift it towards right.
- Rule 5 – If there is difficulty with summing point while simplifying, shift it towards left.
- Rule 6 – Repeat the above steps till you get the simplified form, i.e., single block.

The term PID stands for proportional integral derivative and it is one kind of device used to control different process variables like pressure, flow, temperature, and speed in industrial applications. In this controller, a control loop feedback device is used to regulate all the process variables.

This type of control is used to drive a system in the direction of an objective location otherwise level. It is almost everywhere for temperature control and used in scientific processes, automation & myriad chemical. In this controller, closed-loop feedback is used to maintain the real output from a method like close to the objective otherwise output at the fixe point if possible. In this article, the PID controller design with control

modes used in them like P, I & D are discussed. A closed-loop system like a PID controller includes a feedback control system. This system evaluates the feedback variable using a fixed point to generate an error signal. Based on that, it alters the system output. This procedure will continue till the error reaches Zero otherwise the value of the feedback variable becomes equivalent to a fixed point.

This controller provides good results as compared with the ON/OFF type controller. In the ON/OFF type controller, simply two conditions are obtainable to manage the system. Once the process value is lower than the fixed point, then it will turn ON. Similarly, it will turn OFF once the value is higher than a fixed value. The output is not stable in this kind of controller and it will swing frequently in the region of the fixed point. However, this controller is more steady & accurate as compared to the ON/OFF type controller.

With the use of a low cost simple ON-OFF controller, only two control states are possible, like fully ON or fully OFF. It is used for a limited control application where these two control states are enough for the control objective. However oscillating nature of this control limits its usage and hence it is being replaced by PID controllers.

PID controller maintains the output such that there is zero error between the process variable and setpoint/ desired output by closed-loop operations. PID uses three basic control behaviors that are explained below.

### **P- Controller**

Proportional or P- controller gives an output that is proportional to current error  $e(t)$ . It compares the desired or set point with the actual value or feedback process value. The resulting error is multiplied with a proportional constant to get the output. If the error value is zero, then this controller output is zero.

### **I-Controller**

Due to the limitation of p-controller where there always exists an offset between the process variable and setpoint, I-controller is needed, which provides necessary action to eliminate the steady-state error. It integrates the error over a period of time until the error value reaches zero. It holds the value to the final control device at which error becomes zero. Integral control decreases its output when a negative error takes place. It limits the speed of response and affects the stability of the system. The speed of the response is increased by decreasing integral gain,  $K_i$ .

### **D-Controller**

I-controller doesn't have the capability to predict the future behavior of error. So it reacts normally once the setpoint is changed. D-controller overcomes this problem by anticipating the future behavior of the error. Its output depends on the rate of change of error with respect to time, multiplied by derivative constant. It gives the kick start for the output thereby increasing system response.

**In the simulation of generated transfer function the auto tune of PID control toolbox resulted in a very good result, although the self tuning was used to reduce the overshoot and peak time, but it seems that matlab optimised value are the most suitable for such high order systems**